CV Sorting using LLMs (Capstone_Project-CS[02])
Student ID: m0b0xhj

Abstract
This project implements an automated CV ranking system that scores candidate resumes against a job description using two free local large language models (llama3.2 and gemma2) served by Ollama. Resumes in TXT/PDF/DOCX are parsed into structured text, each model produces an evidence-based fit score with strengths and gaps, and the scores are aggregated into a transparent ranked list. The design is modular and plug-and-play: LLM providers, parsers, and ranking logic are separable. A mock provider enables offline testing of edge cases without a GPU.

1. Introduction
Recruiters spend significant time manually comparing resumes to role requirements. Keyword filters miss context, while closed commercial APIs raise cost and privacy concerns. This capstone builds a terminal-first ranking pipeline using open models that can run locally.

2. Problem Statement
Given a job description and a set of candidate CVs, produce a ranked list with explainable match scores that reflect skills, experience, education, and responsibility alignment.

3. Objectives
- Parse CVs and job descriptions from common document formats.
- Use at least two LLMs (llama3.2 + gemma2) with justified dual scoring.
- Aggregate model outputs into a stable ranked list with explanations.
- Keep the codebase simple, testable, and free of LangChain complexity.
- Support plug-and-play LLM backends and offline mock evaluation.

4. Methodology
Tools: Python 3, Ollama, pypdf, requests. No LangChain.
Workflow: parse JD -> parse CVs -> prompt each LLM -> parse JSON scores -> weighted aggregate -> rank -> export JSON + console table.
Modules: config.py, parsers.py, prompts.py, llm_client.py (OllamaProvider / MockLLMProvider), matcher.py, ranker.py, main.py.
Model justification: llama3.2 provides strong general instruction following for structured JSON scoring; gemma2 adds a second open-weight judgment to reduce single-model bias. Equal weights (0.5/0.5) keep aggregation interpretable.

5. Results and Analysis
On the sample corpus (Alice/Bob/Cara vs Senior Python Backend JD), mock dual-model ranking places Python/backend-aligned profiles above a Java/finance profile. With --required-skills kubernetes, Cara ranks first due to explicit Kubernetes evidence. Unit tests cover 40+ cases: empty/corrupt files, invalid LLM JSON, provider failures, tie-breaking, and skill-filter refinement.

6. Conclusion

The system meets CS[02] goals with a reliable, free, local dual-LLM ranking pipeline. Extensions include richer embedding pre-filters, calibrated score fusion, and optional UI dashboards while preserving the terminal entrypoint.